Homework:

Question: How do HTTP applications preserve state across request-response cycles for user authentication and session management?

Answer:

Because HTTP is a stateless protocol, web applications cannot natively retain user information between independent request-response cycles. To overcome this limitation and maintain continuity—such as keeping a user logged in—applications rely on client-side state storage mechanisms, primarily session cookies and authentication tokens.

In session-based management, once a user authenticates, the server generates a unique session identifier and transmits it to the browser via a secure Set-Cookie response header. The browser automatically attaches this session cookie to all subsequent requests to the same domain. Upon receiving a request, the server extracts the cookie, retrieves the corresponding session state from a database or cache (like Redis), and re-establishes the user's authenticated context.

Alternatively, token-based authentication uses self-contained tokens, such as JSON Web Tokens (JWTs). Following authentication, the server generates a digitally signed token containing user claims and sends it back to the client. The browser stores this token and sends it in the Authorization header of future requests. Rather than querying a central session database on every cycle, the server cryptographically verifies the incoming token's signature to validate identity, effectively preserving application state in a scalable, decoupled manner

Question: What are the procedures for performing Django database migrations to a server-based relational database like MariaDB?

Answer:

Migrating a Django application from a local database to a server-based relational database like MariaDB involves database provisioning, driver installation, settings configuration, and schema migration.

First, the MariaDB server must be configured by creating a dedicated database, user account, and granting full schema privileges. Next, an appropriate database adapter, such as mysqlclient or PyMySQL, is installed in the Python environment to enable connectivity. In settings.py, the DATABASES dictionary is modified to replace the default engine with django.db.backends.mysql, providing the remote database NAME, USER, PASSWORD, HOST, and PORT.

Once the network connection is established, Django's two-step migration system builds the production schema. Running python manage.py makemigrations scans models.py files to construct or update Python migration scripts. Executing python manage.py

migrate then translates these scripts into raw SQL DDL statements and applies them directly to the remote MariaDB instance, creating tables, foreign key constraints, and indexes. Finally, running python manage.py createsuperuser or loaddata populates the initial administrative data, completing the production deployment.